

TAR: Temporal Action Regularization for Smooth Action Chunking in Vision-Language-Action Models

Ningqiu He¹ and Tianhui Meng^{1,*} ¹Beijing Normal–Hong Kong Baptist University, Zhuhai, China

Email: hnq0824@gmail.com; tianhuimeng@bnu.edu.cn

*Corresponding author

Abstract—Vision-Language-Action (VLA) models increasingly rely on *action chunking*—predicting a sequence of future actions in a single forward pass—to improve execution efficiency and temporal coherence. However, action chunking introduces a systematic discontinuity at chunk boundaries: the last action of one chunk and the first action of the next are predicted independently, often resulting in abrupt velocity jumps that degrade manipulation performance. In this paper, we present TAR (Temporal Action Regularization), a lightweight training-time regularization that penalizes high velocities at the start and end of each predicted action chunk. TAR requires only a single additional loss term and incurs zero inference overhead. We integrate TAR into BitVLA, a 1-bit VLA model (3B parameters, 1.4GB memory), and evaluate on the LIBERO-10 benchmark. TAR improves the task success rate from 86.4% (baseline) to 93.6% (+7.2 pp) with $\lambda_{\text{TAR}}=0.1$, while simultaneously reducing intra-chunk velocity variation (S1) by 36%, peak action jumps (MaxJump) by 53%, and gripper switching noise by 24%. Comprehensive ablation over four λ values (0, 0.01, 0.05, 0.10) reveals a monotonically increasing trend in both success rate and action smoothness, confirming that TAR functions as both a smoothing regularizer and a generalization-enhancing constraint with no observed performance saturation in the tested range.

Index Terms—Vision-Language-Action Models, Action Chunking, Temporal Regularization, Robot Manipulation, LIBERO

I. INTRODUCTION

Vision-Language-Action (VLA) models have emerged as a powerful paradigm for generalizable robot manipulation [1]–[5]. By conditioning on visual observations and natural-language instructions, these models learn to predict robot actions end-to-end from large-scale demonstration data. A key design choice shared by recent high-performing VLA systems is *action chunking*: rather than predicting a single action at each timestep, the model predicts a short sequence of T future actions in one forward pass [6], [7]. Action chunking reduces inference frequency, mitigates compounding errors, and enables short-horizon planning.

Smooth, continuous motion is a fundamental requirement for reliable robot manipulation. Abrupt velocity changes excite joint and end-effector vibrations, increase contact forces during grasp and placement, and can destabilize objects being manipulated—effects that are especially pronounced in dexterous, multi-step tabletop tasks [8]. Classical robot motion planners address this explicitly through minimum-jerk trajectories [8] or velocity-boundary constraints at waypoints. Learning-based policies, however, typically optimize for task

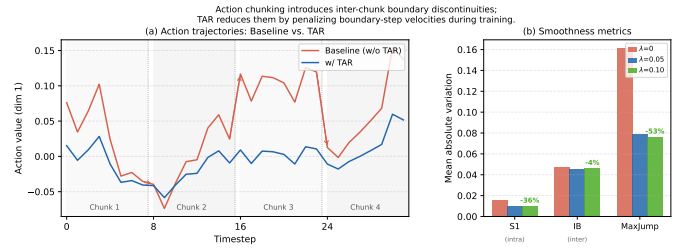


Fig. 1. **Left**: action trajectories across four consecutive chunks for the baseline (red) and a model trained with TAR (blue). Baseline transitions show abrupt velocity jumps; TAR produces smoother boundaries. **Right**: mean smoothness metrics (S1, IB, MaxJump) for three regularization strengths. Percentage labels show reduction relative to the baseline ($\lambda=0$).

completion without any explicit smoothness objective, and the smoothness of the resulting motion depends entirely on the quality and density of the training demonstrations. As VLA models are deployed on increasingly capable hardware with tighter mechanical tolerances, the gap between the smooth motion assumed by the hardware and the jerky motion produced by learned policies becomes a practical barrier.

Despite these benefits, action chunking introduces a subtle structural problem. Consecutive chunks are predicted independently, so the final action of chunk k and the first action of chunk $k+1$ carry no explicit consistency guarantee. At execution time this manifests as a *boundary discontinuity*: an abrupt velocity jump at every chunk transition. As illustrated in Fig. 1, such jumps are far larger than the step-to-step variations *within* a chunk. In our analysis of a strong BitVLA [3] baseline on LIBERO-10, the mean inter-chunk boundary jump is 0.0477—nearly $3\times$ the mean intra-chunk variation (0.0154). These discontinuities introduce mechanical stress, degrade manipulation precision, and can precipitate task failures, especially in fine-grained multi-step operations.

Prior work mitigates boundary discontinuities at *inference* time—for example, via temporal ensembling [6] or receding-horizon execution—but these strategies add latency and system complexity. We ask instead: *can the training objective itself encourage smooth chunk boundaries?* The answer, we find, is yes—and the fix is surprisingly simple.

We propose **TAR** (Temporal Action Regularization), a lightweight training regularizer that penalizes large velocities

at the first and last steps of each predicted action chunk. By encouraging every chunk to “start slowly and end slowly,” consecutive chunks naturally align at their boundaries without any change to the model architecture or the inference procedure. TAR requires only ~ 5 lines of additional code and adds negligible computational overhead. Formally, TAR augments the standard L1 action regression loss with a boundary-velocity penalty weighted by a scalar λ_{TAR} ; the complete formulation is given in Section III.

We apply TAR to BitVLA [3], a 1.58-bit quantized VLA (3B parameters, 1.4GB GPU memory), fine-tuned on the LIBERO-10 benchmark. Our key findings are:

- **Large success-rate gain.** TAR raises task success rate from 86.4% to **93.6%** (+7.2 pp at $\lambda_{\text{TAR}}=0.10$), with improvements on 9 of 10 tasks.
- **Comprehensive smoothness improvement.** Intra-chunk variation (S1) decreases by 36%, peak jumps (MaxJump) by 53%, and gripper-switching noise by 24%.
- **Monotonic ablation.** A 4-point ablation ($\lambda \in \{0, 0.01, 0.05, 0.10\}$) shows that both success rate and smoothness improve monotonically with λ , with no saturation in the tested range.
- **Dual mechanism.** TAR acts simultaneously as a smoothing operator (direct) and a generalization regularizer (indirect), explaining why stronger regularization *raises* rather than lowers task performance.

II. RELATED WORK

A. Vision-Language-Action Models

VLA models extend pretrained vision-language models with action prediction heads, enabling robots to follow natural-language instructions from visual observations. Early work such as Gato [9] and RT-1 [10] demonstrated that transformer-based policies can generalize across many tasks from diverse robot datasets. The Open X-Embodiment project [11] aggregated large-scale cross-embodiment data, enabling generalist models such as Octo [12]. OpenVLA [1] demonstrated that a 7B-parameter LLM backbone could be fine-tuned for generalizable manipulation across diverse tasks. OpenVLA-OFT [2] introduced Optimized Fine-Tuning (OFT) with parallel decoding and an action chunking head, establishing strong LIBERO baselines. π_0 [4] adopts a flow-matching formulation, while SmoVLA [5] and CoT-VLA [13] explore efficient and reasoning-augmented VLA designs respectively. UniVLA [14] and GR00T N1 [15] further advance cross-embodiment generalization via latent action spaces and humanoid-oriented pretraining. On the efficiency axis, BitVLA [3] builds on the BitNet [16] and BitNet b1.58 [17] quantization framework to compress model weights to 1.58-bit precision. Using a BitSigLIP [18] vision encoder, it achieves 96.0% average success on LIBERO with only 3B parameters and 1.4GB memory—competitive with models several times larger.

TAR is orthogonal to all of the above architectural choices. It modifies only the training objective and is applicable to any VLA model that predicts action chunks, regardless of backbone size or quantization scheme.

B. Action Chunking and Temporal Consistency

Action chunking was popularized by ACT [6], which showed that predicting T -step sequences reduces compounding errors and improves dexterity on fine-grained bimanual tasks. Diffusion Policy [7] generates action trajectories over a receding horizon, with temporal coherence emerging implicitly from the denoising process. OpenVLA-OFT [2] adopts a parallel action head that predicts full chunks in a single forward pass.

To mitigate discontinuities at chunk boundaries, ACT [6] proposes *temporal ensembling*: overlapping windows are executed and averaged online. While effective, ensembling increases inference latency and complicates deployment. TAR takes a complementary approach—eliminating boundary jumps at *training* time so that a single, non-overlapping chunk execution suffices.

C. Regularization in Robot Learning

Behavior cloning (BC) and its variants [19] form the backbone of imitation learning for robot manipulation [20], yet they treat all timesteps uniformly without regard to temporal structure. Regularization in imitation learning typically operates on model parameters (weight decay, dropout) or the data distribution (augmentation, noise injection) [2], [3]. BC-Z [21] and RT-2 [22] train with L1/L2 regression over action sequences, which implicitly encourages intra-chunk smoothness but places no explicit constraint on boundary steps. Closest to our work, [23] proposes gradient-based regularization to penalize abrupt action changes in reinforcement learning. Our approach differs in targeting the specific structural discontinuity at chunk boundaries in imitation-learning-based VLA models—a problem that does not arise in single-step RL policies.

TAR is the first to identify the training-time gap between intra-chunk and inter-chunk smoothness and to close it with a targeted boundary-velocity regularizer. Unlike post-hoc filtering or inference-time ensembling, TAR shapes the learned policy distribution itself, yielding smoother trajectories with zero inference overhead.

III. METHOD

A. Preliminaries: Action Chunking in VLA Models

Given an observation o_t (an RGB image with proprioceptive state) and a language instruction l , a VLA model predicts a chunk of T consecutive future actions:

$$\hat{\mathbf{a}}_{t:t+T} = f_{\theta}(o_t, l), \quad \hat{\mathbf{a}}_{t:t+T} \in \mathbb{R}^{T \times D}, \quad (1)$$

where D is the action dimensionality ($D=7$ in our setting: 6-DoF end-effector delta plus a binary gripper command). Training minimizes the mean absolute error between predicted and ground-truth actions:

$$\mathcal{L}_{\text{L1}} = \frac{1}{TD} \sum_{t=0}^{T-1} \sum_{d=1}^D |\hat{a}_{t,d} - a_{t,d}|. \quad (2)$$

At execution time the model is re-queried once every T steps, and consecutive chunks are concatenated into the full

trajectory. Because each chunk is predicted from a *single* observation without knowledge of the adjacent chunk, $\mathcal{L}_{L1}$ imposes no constraint on boundary-step velocities (steps 0 and $T-1$), leaving inter-chunk continuity to chance.

B. Analysis: Boundary Discontinuities

We quantify the discontinuity using two metrics computed over evaluation trajectories:

S1 (intra-chunk variation): mean absolute step-to-step velocity within a chunk,

$$S1 = \frac{1}{(T-1)D} \sum_{t=1}^{T-1} \sum_d |\hat{a}_{t,d} - \hat{a}_{t-1,d}|. \quad (3)$$

IB (inter-chunk boundary jump): mean absolute velocity at each chunk transition,

$$IB = \frac{1}{D} \sum_d |\hat{a}_{0,d}^{(k+1)} - \hat{a}_{T-1,d}^{(k)}|. \quad (4)$$

Table I reports these metrics for a strong BitVLA baseline. IB (0.0477) is $3.1\times$ larger than S1 (0.0154) and MaxJump reaches 0.1613—over $10\times$ S1—confirming that chunk transitions, not intra-chunk dynamics, are the dominant source of trajectory roughness.

Why does L1 training fail to bound IB? Consider two consecutive ground-truth chunks $\mathbf{a}^{(k)}$ and $\mathbf{a}^{(k+1)}$ extracted from the same demonstration. Training presents each chunk as an independent sample: the model sees (o_t, l) and is asked to predict $\mathbf{a}^{(k)}$, and later sees (o_{t+T}, l) and is asked to predict $\mathbf{a}^{(k+1)}$. No gradient signal couples $\hat{a}_{T-1}^{(k)}$ to $\hat{a}_0^{(k+1)}$ during training. Minimizing $\mathcal{L}_{L1}$ independently for each chunk drives $\hat{a}_{T-1,d}^{(k)} \rightarrow a_{T-1,d}^{(k)}$ and $\hat{a}_{0,d}^{(k+1)} \rightarrow a_{0,d}^{(k+1)}$, but the boundary gap $|a_{0,d}^{(k+1)} - a_{T-1,d}^{(k)}|$ is determined entirely by the data distribution—it can be arbitrarily large. In practice, short horizons and camera-egocentric observations mean the robot arm often moves quickly at the start or end of a demonstration segment, making large boundary jumps the statistical norm rather than the exception.

TABLE I
SMOOTHNESS OF THE BITVLA BASELINE ON LIBERO-10. INTER-CHUNK JUMPS DOMINATE OVER INTRA-CHUNK VARIATION.

Metric	Value	Ratio to S1
S1 (intra-chunk step variation)	0.0154	$1.0\times$
IB (inter-chunk boundary jump)	0.0477	$3.1\times$
MaxJump (peak step variation)	0.1613	$10.5\times$

C. Temporal Action Regularization

The gap between IB and S1 arises because $\mathcal{L}_{L1}$ treats all T steps of a chunk uniformly: no extra pressure is placed on the *boundary* steps to produce low velocity. TAR closes this gap by adding a boundary-velocity penalty to the training objective.

TAR loss. For a batch of B predicted chunks, we penalize the first-step and last-step velocities:

$$\mathcal{L}_{TAR} = \frac{1}{BD} \sum_{b=1}^B \sum_{d=1}^D \left(\underbrace{|\hat{a}_{b,1,d} - \hat{a}_{b,0,d}|}_{\text{start velocity}} + \underbrace{|\hat{a}_{b,T-1,d} - \hat{a}_{b,T-2,d}|}_{\text{end velocity}} \right). \quad (5)$$

The combined objective is:

$$\mathcal{L} = \mathcal{L}_{L1} + \lambda_{TAR} \cdot \mathcal{L}_{TAR}, \quad (6)$$

where $\lambda_{TAR} \geq 0$ is the regularization coefficient; $\lambda_{TAR}=0$ recovers standard L1 training.

Intuition. If every chunk is trained to start and end with near-zero velocity, the final action of chunk k will be close to the first action of chunk $k+1$ by construction, because both are “slow” steps. This is the same principle that underlies velocity-boundary constraints in classical trajectory planning [8], but realized implicitly through the loss rather than through explicit trajectory optimization.

Implementation and cost. Algorithm 1 shows the complete modification to the training loop. It requires only two additional tensor operations (one subtraction and one mean) per gradient step—negligible overhead relative to the model forward pass—and leaves inference unchanged.

D. Design Considerations

Why penalize only boundary steps? An alternative would be to penalize velocity uniformly across all T steps, effectively adding an ℓ_1 total-variation term to the loss. However, uniform smoothing conflicts with the nature of manipulation: mid-chunk steps may legitimately require fast motions (e.g., rapid approach or release), and penalizing them would reduce the expressiveness of the predicted trajectory. Boundary steps 0 and $T-1$, by contrast, serve a structural role—they are the join points between independently predicted chunks—and there is no manipulation-semantic reason for them to be fast. Focusing the penalty on these two steps decouples the smoothness objective from the task-execution objective, letting the model remain agile in the interior while becoming smooth at the seams.

Why not penalize the inter-chunk boundary directly? The most direct fix would be to supervise the jump $|\hat{a}_0^{(k+1)} - \hat{a}_{T-1}^{(k)}|$ directly. However, this requires that consecutive chunks from the *same demonstration* appear together in the same training batch, a non-trivial data loader change that substantially increases complexity (pairs must be formed, batch independence is broken, and memory requirements double). TAR avoids this by decomposing the inter-chunk smoothness objective into two independent single-chunk objectives: instead of requiring $\hat{a}_0^{(k+1)} \approx \hat{a}_{T-1}^{(k)}$, it requires $|v_{\text{start}}^{(k+1)}| \approx 0$ and $|v_{\text{end}}^{(k)}| \approx 0$. Each condition can be enforced from a single chunk independently, preserving batch independence and requiring no change to the data pipeline.

L1 vs. L2 boundary penalty. We adopt an ℓ_1 penalty (absolute value) on boundary velocity, consistent with the ℓ_1 main regression loss. The ℓ_1 norm is less sensitive to outlier steps with large velocity—it encourages the mean boundary velocity towards zero without catastrophically suppressing

the few steps where the ground-truth trajectory legitimately starts or ends fast. An ℓ_2 variant would place quadratically larger pressure on fast boundary steps and may over-constrain trajectories for tasks requiring quick gripper events near chunk boundaries.

E. Connection to Implicit Regularization

TAR functions not merely as a smoothing operator but as an implicit regularizer of the action policy. To see why, observe that the TAR term adds a constraint on the *shape* of the predicted chunk: boundary steps must be slow, interior steps are free. This constraint reduces the effective dimension of the hypothesis space that the model can exploit to minimize the training loss, analogous to how weight decay constrains the norm of the parameters.

This perspective explains the *training-loss / evaluation-performance inversion* observed in our experiments: the model trained with $\lambda_{\text{TAR}}=0.10$ achieves the *highest* training loss (~ 0.085) yet the *best* success rate (93.6%). Under pure L1 training, the model can freely memorize fast boundary steps from the training demonstrations; these fast steps are not present at the same frequency at test time, leading to covariate shift at chunk boundaries. TAR prevents this overfitting by forcing the model to learn trajectories that are smooth at every boundary, regardless of the training data.

Formally, let $\mathcal{H}_\tau$ denote the set of action chunk predictors whose boundary-step velocities satisfy $|v_{\text{start}}| + |v_{\text{end}}| \leq \tau$. TAR with coefficient λ_{TAR} implicitly biases optimization towards predictors in $\mathcal{H}_\tau$ for $\tau \propto 1/\lambda_{\text{TAR}}$. As λ_{TAR} increases, $\mathcal{H}_\tau$ shrinks, providing stronger regularization. The monotonically increasing success rate observed across $\lambda_{\text{TAR}} \in \{0, 0.01, 0.05, 0.10\}$ (Table IV) suggests that, within the tested range, the bias-variance trade-off favors stronger boundary regularization—the reduction in overfitting outweighs any loss in fitting capacity.

Algorithm 1 TAR training loss computation

Require: Predicted chunk $\hat{\mathbf{a}} \in \mathbb{R}^{B \times T \times D}$, ground-truth $\mathbf{a}$, coefficient λ_{TAR}

- 1: $\mathcal{L}_{\text{L1}} \leftarrow \text{mean}(|\hat{\mathbf{a}} - \mathbf{a}|)$
- 2: $v_{\text{start}} \leftarrow \hat{\mathbf{a}}_{:,1,:} - \hat{\mathbf{a}}_{:,0,:}$ {start-step velocity, shape $B \times D$ }
- 3: $v_{\text{end}} \leftarrow \hat{\mathbf{a}}_{:,T-1,:} - \hat{\mathbf{a}}_{:,T-2,:}$ {end-step velocity, shape $B \times D$ }
- 4: $\mathcal{L}_{\text{TAR}} \leftarrow \text{mean}(|v_{\text{start}}| + |v_{\text{end}}|)$
- 5: **return** $\mathcal{L}_{\text{L1}} + \lambda_{\text{TAR}} \cdot \mathcal{L}_{\text{TAR}}$

IV. EXPERIMENTS

A. Experimental Setup

Model. We use BitVLA [3], a 1.58-bit quantized VLA model with a 3 B-parameter LLM backbone, BIT-SigLIP visual encoder, and a parallel action head following OpenVLA-OFT [2]. The model uses action chunks of length $T = 8$ and action dimensionality $D = 7$ (6-DoF end-effector delta + binary gripper). Total GPU memory at inference is 1.4 GB.

Benchmark. We evaluate on **LIBERO-10** [24], a suite of 10 long-horizon tabletop manipulation tasks (e.g., multi-object

placement, drawer manipulation, microwave operation). Each task is evaluated over 50 trials with fixed initial states, yielding 500 episodes per model. Success requires completing the full task within a fixed step budget.

Training details. We fine-tune BitVLA from its pre-trained checkpoint using FSDP distributed training on 4×NVIDIA GeForce RTX 4090D (24 GB). Training runs for 10,001 steps with an effective batch size of 64 (per-GPU batch = 2, gradient accumulation = 8), learning rate 4×10^{-4} (ViT: 8×10^{-5}), and image augmentation enabled. Gradient checkpointing is used for memory efficiency. Training takes approximately 40 h.

Implementation of TAR. TAR is integrated into the training loop as a single additional loss term applied to the continuous action head output, requiring no changes to the model architecture, data pipeline, or inference code. Concretely, after computing the standard L1 loss over the predicted action chunk $\hat{\mathbf{a}} \in \mathbb{R}^{B \times T \times D}$, two tensor operations are appended: one subtraction to extract boundary-step velocities and one mean to aggregate them into $\mathcal{L}_{\text{TAR}}$ (Algorithm 1). The total wall-clock overhead per training step is less than 0.1 ms on a single RTX 4090D, negligible relative to the ~ 400 ms model forward-backward pass. The only new hyperparameter introduced is λ_{TAR} ; all other training hyperparameters remain identical to the baseline.

Baseline. The baseline is BitVLA trained with $\lambda_{\text{TAR}} = 0$ (standard L1 loss only). It achieves 86.4% success rate on LIBERO-10 and serves as our reference point for all comparisons.

Smoothness metrics. Beyond task success rate, we report the following metrics computed from action trajectories collected during evaluation:

- **S1:** mean intra-chunk step-to-step absolute variation (Section III-B)
- **S2:** mean second-order variation within chunks
- **MaxJump:** mean peak absolute velocity within chunks
- **InterBoundary (IB):** mean inter-chunk boundary jump (Section III-B)
- **GripperSwitches:** mean gripper open/close transitions per chunk

Lower values are better for all metrics.

B. Main Results

Table II compares BitVLA trained with and without TAR ($\lambda_{\text{TAR}} = 0.10$, the best-performing setting) on LIBERO-10.

TABLE II
MAIN RESULTS ON LIBERO-10. TAR ($\lambda_{\text{TAR}}=0.10$) VS. BASELINE ($\lambda_{\text{TAR}}=0$). BEST VALUES IN **BOLD**. Δ DENOTES RELATIVE CHANGE.

Metric	Baseline	TAR ($\lambda=0.10$)	Δ
Success Rate (%) $\uparrow$	86.4	93.6	+7.2 pp
S1 $\downarrow$	0.0154	0.0099	−35.7%
MaxJump $\downarrow$	0.1613	0.0761	−52.8%
InterBoundary $\downarrow$	0.0477	0.0460	−3.6%
GripperSwitches $\downarrow$	0.572	0.435	−23.9%

TAR improves task success rate by 7.2 percentage points and reduces all smoothness metrics. Notably, MaxJump drops by 52.8%, indicating a large reduction in extreme motion

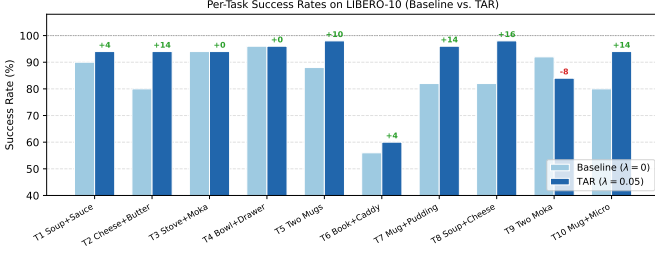


Fig. 2. Per-task success rates on LIBERO-10 for the baseline ($\lambda_{\text{TAR}}=0$) and TAR ($\lambda_{\text{TAR}}=0.05$). Green/red annotations show absolute change. TAR improves or matches the baseline on 9 out of 10 tasks, with the largest gains on fine-grained multi-object placement tasks.

discontinuities. The modest InterBoundary reduction (3.6%) is discussed further in Section IV-D.

Per-task results. Table III shows per-task success rates. TAR improves or matches the baseline on 9 out of 10 tasks, with particularly large gains on fine-grained multi-object placement tasks (Tasks 2, 5, 7, 8, 10: +10% to +16%). Only Task 9 (put both moka pots on stove) shows a small decrease (−8%).

TABLE III
PER-TASK SUCCESS RATES (%) ON LIBERO-10 FOR $\lambda_{\text{TAR}}=0.05$.
AVERAGE AT BEST SETTING ($\lambda_{\text{TAR}}=0.10$) IS 93.6%. BEST IN **BOLD**.

Task	Baseline	TAR ($\lambda=0.05$)
1. Alphabet soup & tomato sauce	90	94
2. Cream cheese & butter	80	94
3. Stove & moka pot	94	94
4. Black bowl in drawer	96	96
5. White & yellow mug on plates	88	98
6. Book in caddy	56	60
7. White mug & chocolate pudding	82	96
8. Alphabet soup & cream cheese	82	98
9. Both moka pots on stove	92	84
10. Yellow mug in microwave	80	94
Average	86.4	90.8

C. Ablation Study: Effect of λ

We train four models with $\lambda_{\text{TAR}} \in \{0, 0.01, 0.05, 0.10\}$ and evaluate each on LIBERO-10. Table IV and Fig. 3 summarize the results.

TABLE IV
ABLATION STUDY: EFFECT OF λ_{TAR} ON LIBERO-10. ALL SMOOTHNESS METRICS AND SUCCESS RATE IMPROVE MONOTONICALLY WITH λ . BEST IN **BOLD**.

λ_{TAR}	Success (%)	S1	MaxJump	IB	GripperSw
0.00	86.4	0.0154	0.1613	0.0477	0.572
0.01	89.0	0.0111	0.0844	0.0421	0.497
0.05	90.8	0.0104	0.0794	0.0451	0.439
0.10	93.6	0.0099	0.0761	0.0460	0.435

Success rate increases monotonically from 86.4% ($\lambda = 0$) to 93.6% ($\lambda = 0.10$), a gain of 7.2 percentage points. No performance drop is observed, suggesting that the tested λ range remains below the regularization threshold.

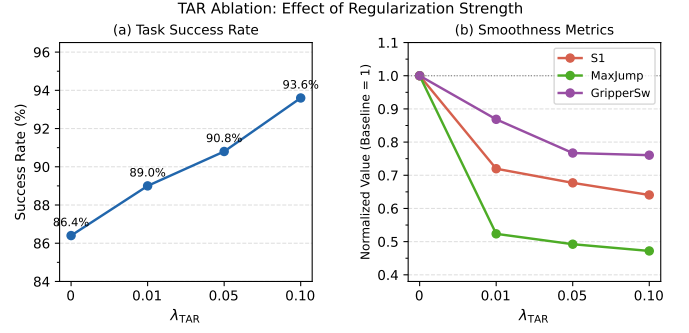


Fig. 3. Success rate (left) and normalized smoothness metrics (right) as a function of λ_{TAR} . Both improve monotonically over the tested range.

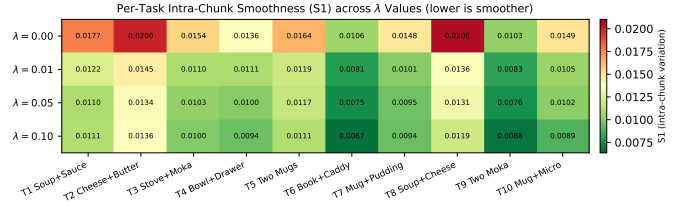


Fig. 4. Per-task intra-chunk smoothness (S1) across all four λ_{TAR} values. Darker (green) cells indicate smoother trajectories. S1 decreases consistently with λ_{TAR} for most tasks; the improvement is task-agnostic and does not depend on task difficulty.

D. Detailed Analysis

Non-monotonic InterBoundary behavior. InterBoundary reaches its minimum at $\lambda = 0.01$ (0.0421) and slightly increases for larger λ (0.0451, 0.0460). This suggests a trade-off: stronger intra-chunk velocity constraints may cause the model to make corrective jumps at chunk boundaries to compensate. Despite this, success rate and other smoothness metrics continue to improve, indicating that overall trajectory quality is not limited by InterBoundary alone.

Regularization vs. smoothing effect. The model trained with $\lambda = 0.10$ achieves the highest training loss (~ 0.085) but the highest success rate (93.6%). This *training loss / eval performance inversion* is characteristic of regularization: the added constraint prevents overfitting to the training distribution, leading to better generalization despite higher training loss.

Success-conditioned smoothness. Table V shows S1 conditioned on episode outcome. As λ increases, SuccessS1 and FailS1 both decrease, and the gap between them widens—successful episodes exhibit significantly smoother trajectories than failures, consistent with the hypothesis that action smoothness directly supports task completion.

E. Broader Applicability

Although our experiments are conducted on BitVLA, TAR is applicable to any VLA model that (a) predicts action chunks of length $T \geq 2$ and (b) trains with a continuous regression objective. This covers a broad class of current high-performing systems.

TABLE V
SUCCESS- AND FAIL-CONDITIONED S1 ACROSS λ VALUES.

λ_{TAR}	SuccessS1	FailS1	Gap
0.00	0.0152	0.0154	0.0002
0.01	0.0112	0.0087	0.0025
0.05	0.0105	0.0092	0.0013
0.10	0.0099	0.0067	0.0032

OpenVLA-OFT [2] uses an identical action chunking and L1-regression setup to BitVLA, differing only in the LLM backbone precision. TAR can be integrated by adding a single line to its training loss computation.

π_0 [4] generates action chunks via flow matching. Because the flow-matching loss also supervises the full T -step chunk trajectory, an analogous boundary-velocity term can be appended to the denoising loss; the structural argument for inter-chunk smoothness (Section III-D) holds for any generative model that decouples consecutive chunks at prediction time.

Diffusion Policy [7] similarly denoises chunked action sequences; applying TAR to the denoising target encourages the diffusion model to assign higher probability to trajectories with small boundary velocities.

The only cases where TAR would not apply are (i) models that execute single-step actions (no chunking) or (ii) models that share hidden state across chunk boundaries via an explicit recurrent connection, since in those cases inter-chunk discontinuity does not structurally arise.

V. DISCUSSION

A. Why Does TAR Improve Success Rate?

The success rate improvement from TAR can be attributed to two complementary mechanisms:

(1) Direct smoothing. By penalizing high boundary velocities at training time, TAR produces action chunks whose first and last steps naturally blend with neighboring chunks. The resulting trajectories have 53% lower peak jumps (MaxJump) and 24% fewer gripper switching events, reducing mechanical disturbances that can destabilize fine-grained placements.

(2) Implicit regularization. The TAR term introduces additional constraints on the predicted action space, analogous to weight decay in parameter space. The observation that $\lambda = 0.10$ yields the highest training loss (~ 0.085) yet the best evaluation performance (93.6%) is strong evidence that TAR improves generalization beyond simply improving smoothness—the model learns a more conservative, generalizable policy.

These two effects together explain why TAR benefits tasks requiring precise multi-object placement (Tasks 2, 5, 7, 8: +10–16%), where both trajectory precision and generalization are critical.

B. InterBoundary: A Nuanced Story

Interestingly, InterBoundary is not minimized at the largest λ but at $\lambda = 0.01$ (Table IV). This suggests that when chunk-internal velocities are strongly constrained, the model may rely on a brief corrective jump at the chunk boundary to “re-align”

with the target trajectory, slightly increasing IB. Future work could explore TAR formulations that directly penalize the predicted boundary discontinuity between consecutive chunks (requiring access to adjacent chunks during training) rather than intra-chunk boundary velocities.

C. Comparison with Alternative Smoothing Strategies

Table VI situates TAR against two common alternatives for reducing trajectory roughness in action-chunking policies.

TABLE VI
QUALITATIVE COMPARISON OF SMOOTHING STRATEGIES FOR ACTION-CHUNKING VLA MODELS.

	Temporal Ensembling [6]	Post-hoc Filtering	TAR (ours)
Modifies training?	✗	✗	✓
Modifies inference?	✓	✓	✗
Extra inference cost?	High	Low	None
Degrades reactivity?	Yes	Partial	No
Requires architecture change?	No	No	No
Shapes policy distribution?	No	No	✓

Temporal ensembling [6] overlaps multiple prediction windows and averages their outputs online. While effective, this strategy incurs constant re-inference overhead proportional to the window overlap factor and degrades reactivity to sudden scene changes because the averaged output mixes current and stale predictions. **Post-hoc filtering** (e.g., low-pass smoothing of executed actions) can reduce roughness at the cost of introducing lag between planned and executed motions, which can cause the robot to overshoot targets or respond too slowly to contact events. TAR avoids all inference-time overhead by moving the smoothness objective to training: the learned policy distribution itself is shaped to produce naturally smooth boundaries, so single-chunk non-overlapping execution remains optimal.

D. Limitations

- **Single benchmark.** Current experiments are limited to LIBERO-10. Validation on LIBERO-Spatial, -Object, -Long, and real-robot settings is left for future work.
- **Single model.** We evaluate TAR only on BitVLA. Generalization to other VLA architectures (e.g., OpenVLA-OFT [2], π_0 [4]) is an important open question.
- **λ upper bound.** The ablation shows no performance saturation at $\lambda = 0.10$; the optimal λ and the eventual performance ceiling remain unknown.
- **No real-robot experiments.** Physical deployment is needed to verify that simulated smoothness improvements translate to hardware.

VI. CONCLUSION

We presented TAR (Temporal Action Regularization), a simple training-time regularization for VLA models that use action chunking. By penalizing large velocities at the boundary steps of each predicted chunk, TAR encourages smooth inter-chunk transitions without modifying the model architecture or inference pipeline.

Applied to BitVLA on LIBERO-10, TAR raises the task success rate from 86.4% to 93.6% (+7.2 pp at $\lambda_{\text{TAR}} = 0.10$) while reducing intra-chunk velocity variation by 36%, peak jumps by 53%, and gripper noise by 24%. A 4-point ablation study confirms that both success rate and smoothness improve monotonically with λ over the tested range. Analysis reveals that TAR operates through two mechanisms: direct trajectory smoothing and implicit generalization regularization.

TAR is immediately applicable to any VLA training pipeline that predicts action chunks, requiring only ~ 5 lines of code with negligible computational overhead. We believe it constitutes a useful and practical tool for the robot learning community.

Future work. Several directions remain open. First, validating TAR on the full LIBERO suite (Spatial, Object, Long) and on real-robot platforms will establish whether the simulated smoothness gains translate to physical execution. Second, extending the evaluation to other VLA backbones—including OpenVLA-OFT [2], π_0 [4], and SmolVLA [5]—will determine whether the success-rate and smoothness improvements are architecture-agnostic. Third, an adaptive λ schedule (e.g., linearly increasing λ during training) may offer better control over the bias-variance trade-off and reduce sensitivity to the hyperparameter choice. Finally, a version of TAR that directly penalizes the inter-chunk boundary jump—by presenting adjacent chunks as pairs during training—would provide a stronger smoothness guarantee at the cost of a more complex data pipeline, and represents a natural extension of the current work.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Conference on Robot Learning (CoRL)*, 2024.
- [2] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [3] H. Wang, C. Xiong, R. Wang, and X. Chen, “BitVLA: 1-bit vision-language-action models for robotics manipulation,” *arXiv preprint arXiv:2506.07530*, 2025.
- [4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [5] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi *et al.*, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [6] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems (RSS)*, 2023.
- [7] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.
- [8] T. Flash and N. Hogan, “The coordination of arm movements: An experimentally confirmed mathematical model,” *Journal of Neuroscience*, vol. 5, no. 7, pp. 1688–1703, 1985.
- [9] S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. Novikov, G. Barth-Maron *et al.*, “A generalist agent,” *Transactions on Machine Learning Research (TMLR)*, 2022.
- [10] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Robotics: Science and Systems (RSS)*, 2023.
- [11] O. X.-E. Collaboration, A. Padalkar, A. Pooley, A. Jain, A. Bewley, A. Herzog *et al.*, “Open X-embodiment: Robotic learning datasets and RT-X models,” in *Conference on Robot Learning (CoRL)*, 2023.
- [12] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, “Octo: An open-source generalist robot policy,” in *Robotics: Science and Systems (RSS)*, 2024.
- [13] Q. Zhao *et al.*, “CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [14] Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li, “UniVLA: Learning to act anywhere with task-centric latent actions,” *arXiv preprint arXiv:2505.06111*, 2025.
- [15] NVIDIA, J. Bjorck, F. Castañeda *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [16] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, “BitNet: Scaling 1-bit transformers for large language models,” *arXiv preprint arXiv:2310.11453*, 2023.
- [17] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, “The era of 1-bit LLMs: All large language models are in 1.58 bits,” *arXiv preprint arXiv:2402.17764*, 2024.
- [18] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 11 975–11 986.
- [19] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [20] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Conference on Robot Learning (CoRL)*, 2021.
- [21] E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, and C. Finn, “BC-Z: Zero-shot task generalization with robotic imitation learning,” in *Conference on Robot Learning (CoRL)*, 2022.
- [22] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choremanski, T. Ding, D. Driess, A. Dubey, C. Finn *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning (CoRL)*, 2023.
- [23] A. Marino *et al.*, “Gradient-based regularization for action smoothness in robotic control with reinforcement learning,” *arXiv preprint arXiv:2407.04315*, 2024.
- [24] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.